

Mitigating bandwidth contention at multi-bank LLC in multi-core SOC

Gautambuddha N Meeshi
2019CS10350

December 1, 2022

1 Introduction

The multi-core SOC have multiple cores, multiple LLC banks and directory controller connected by complex interconnection network. To serve the data requests and maintain cache coherency many messages have to be passed between LLC and L2 caches, LLC and directory controller. This leads to bandwidth contention at the LLC. Sakshi, et al. propose a core prioritization algorithm to improve the throughput of the system. This is an extension of work done by Sakshi et al. Previous work used gem5 hardware simulator with classic cache model where single LLC stores all the caches lines. The LLC and L2caches were connected by crossbar interconnection network.

Here we use a more realistic cache model and interconnection network, which is modeled using the Ruby memory subsystem simulator. Ruby includes garnet network simulator.

I describe the simulator details in section 2, system configuration in section 3, the REAL policy in section 4. Section 5 contains the implementation details of REAL in the simulator. Section 6 discusses the experiments performed, result of the experiments and brief analysis of the results. Finally section 7 contains the future work.

2 Simulator details

2.1 gem5

gem5 is an open source computer architecture simulator. It is an event driven simulator i.e., at a given clock cycle (counter) all the events (simulations) that are to be completed at the current clock cycle are popped from event queue, processed, new events are added into the queue and clock cycle is incremented.

2.2 Ruby

Ruby is a detailed simulation model for the memory subsystem. It models inclusive/exclusive cache hierarchies with various replacement policies, coherence protocol implementations, interconnection networks, DMA and memory controllers, various sequencers that initiate memory requests and handle responses [2].

It allows modelling of memory controllers and message objects for the controllers through a language called specification language for implementing cache coherence (SLICC). These controllers encapsulate the simulation objects, viz. cache memory, transaction buffer (MSHR) and message buffers (ports). We must define message structure, cache entry structure, transaction buffer entry structure, states of the cache line, transition blocks and events that should be performed on the transition. A transition is denoted using initial state and the action, it leads to the final state by executing some events.

The entry point into the controllers is the `in_port` function. For each message buffer (port) into the memory controller there is an `in_port` function that reads the message and triggers an appropriate transition.

These memory controllers are connected to the CPU by sequencer and to the dram controller (not part of ruby) through the packet queue object. The ruby memory controllers are connected internally using message buffers and garnet interconnection network.

2.3 Garnet

Garnet is a detailed interconnection network model in gem5. It consists of 6 components, viz. network interface, external link, internal network link, router and flit.

Network interface connects the message buffer from the cache controllers to the garnet network through an external link. External link is connected to a router. The routers are connected one another using internal network link. Router is connected to the network interface using external network link. Flit is the unit of data that is transferred in the network.

The python configuration files can initialize the simulation objects mentioned above to construct the desired interconnection network topology.

2.4 Components of garnet and their working

2.4.1 Network interface

1. Network interface checks the message buffer coming from the controller. Network interface takes out one message from each message buffer, breaks the large message in to 16 bytes chunks called flits and pushes it into an available output virtual channel (VC).
2. There are 12 virtual channel per network link in the network. Each output

virtual channel can hold flits of only one message at a time. Each message buffer can use 4 virtual channels.

3. The network interface iterates over the output virtual channels in round robin order to find a VC with flits, sends 1 flit per clock cycle.
4. Therefore, a data message which is 72 bytes is divided into 5 flits and inserted into a output VC. If there are multiple VCs filled with flits then the flits of messages' will be interleaved while sending because there is only one network link and it chooses the VC in round robin order.
5. The network interface receives the acknowledgement flits from the sender and updates the state of the output VC. An output VC is not marked free until all the acknowledgement of all flits of a message are received.
6. Hence, if the traffic at the network interface is high it will lead to significant increase in network latency of messages.
7. Network interface receives the flits coming from router/NI. Ruby implementation discards all but one flit, all flits contain the pointer to the message. When the tail flit is received, it checks the size of message buffer. If space is available it will push the message into the message buffer and send back the acknowledgement, else it stores it in the stall queue and sends in the next network interface event.

2.4.2 Network link

1. It is responsible for transferring data flits and acknowledgement flits between the component.
2. It checks the source queue and moves the flit to the destination queue. Sets the ready time of the flit to current clock cycle + link latency. An object can not use a message/flit before its ready time.

2.4.3 Router

1. Router internally has input unit for each incoming network link, output unit for each output network link, a switch allocator to redirect the flits from the input unit to correct output unit and crossbar switch to send the flits.
2. **Input unit -**
 - (a) Each input unit contains 12 VC.
 - (b) It consumes one flit per clock cycle.
 - (c) Determines the outport for the flit using the route computation table.
 - (d) Stores the flit at the VC predetermined by the sender. Sender had searched for a free VC and had sent flit for this VC. The free VCs are determined by the acknowledgements received.

3. Switch allocator -

- (a) Iterates over the inports (input units) in round robin order.
- (b) If the output (output is computed by input unit) has a free VC, maps inport to the output.
- (c) For each output it selects an inport in round robin order (multiple inports want to use the output). Pops the flit from the invc, sends acknowledgement to the sender. Pushes the flit into the switch buffer
- (d) At the end of the operation the inport to output mapping is cleared.

4. Crossbar switch -

- (a) Sends the flit to the output by taking out from the switch buffer.

5. Output unit -

- (a) It stores the state of the VCs at the receiving router of a network link.
- (b) It has 12 state variables. Each corresponding to a VC at the receiving router Input unit.
- (c) It receives the acknowledgements sent by the receiver. Acknowledgement also carries a boolean variable that is used to set the out VC state, free.
- (d) The Switch allocator reads these the VC states to determine the free VC at the receiving router of a network link.

I explored the gem5 and ruby by studying the documentation of gem5 and tracing the messages in the controller and flits in the interconnection network using gdb and debug outputs from the gem5.

3 System configuration

This section describes the system configuration that was used for performing the experiments and implementing the REAL policy to the multi bank scenario. There are 8 CPUs each with private L1 (dcache(32KB) and i cache(32KB)), private L2 cache (256KB), 8 independent LLC (1MB each) slices and a directory controller through which DRAM (8GB) requests are sent to the memory controller. Cache line size at all the caches is 64 Bytes. The cachelines are distributed in round robin order among the LLCs.

3.1 Network topology

By default the gem5, garnet uses the crossbar interconnection network. In this topology every controller is connected to a single router. This router routes the flits from source network link to the destination network link.

latency amount of clock cycles and scheduling an event for the controller so that next request is serviced after access latency amount of time. I changed the number of memory requests serviceable per clock cycle to 1 at the L1 cache. Therefore, the L1 cache is non blocking (it can accept a new data request or coherence request each cycle) with access latency of 2 clock cycle. L2 cache is a blocking cache with access latency of 10 CC for data request and 5 clock cycles for coherence request. LLC is a blocking cache with access latency of 30 CC for data request and 15 clock cycles for coherence request.

4 REAL policy

REAL policy uses three metrics to prioritize the cores.

1. **Instructions between successive LLC accesses (ISL)** - This is the number of non LLC instructions (instructions that do not need LLC, viz. ALU instructions, L1 accesses, L2 accesses) between two LLC accesses. This is calculated by taking the ratio of non-LLC instructions to the LLC accesses in a given interval of time. Higher value of ISL is given higher priority.

$$ISL = \#non_LLC_insts / \#LLC_accesses \quad (1)$$

2. **Instruction fetch rate (IFR)** - Number of instruction fetches that reach the LLC among the LLC accesses. A program with high IFR must get high priority so that it does not stall because of not having instructions.

$$IFR = \#I_fetch_LLC / \#LLC_accesses \quad (2)$$

3. **Instruction fetch miss rate (IMR)** - Ratio of instruction fetches that miss the LLC by number of instruction fetch LLC accesses. Low IMR indicates relatively fewer misses among the LLC accesses and could be an indication of compute-intensive applications that merit high priority. Assigning higher priority when the IMR is less than a threshold (IMR_{th}) improves the performance.

$$IMR = \#I_miss_LLC / \#I_fetch_LLC \quad (3)$$

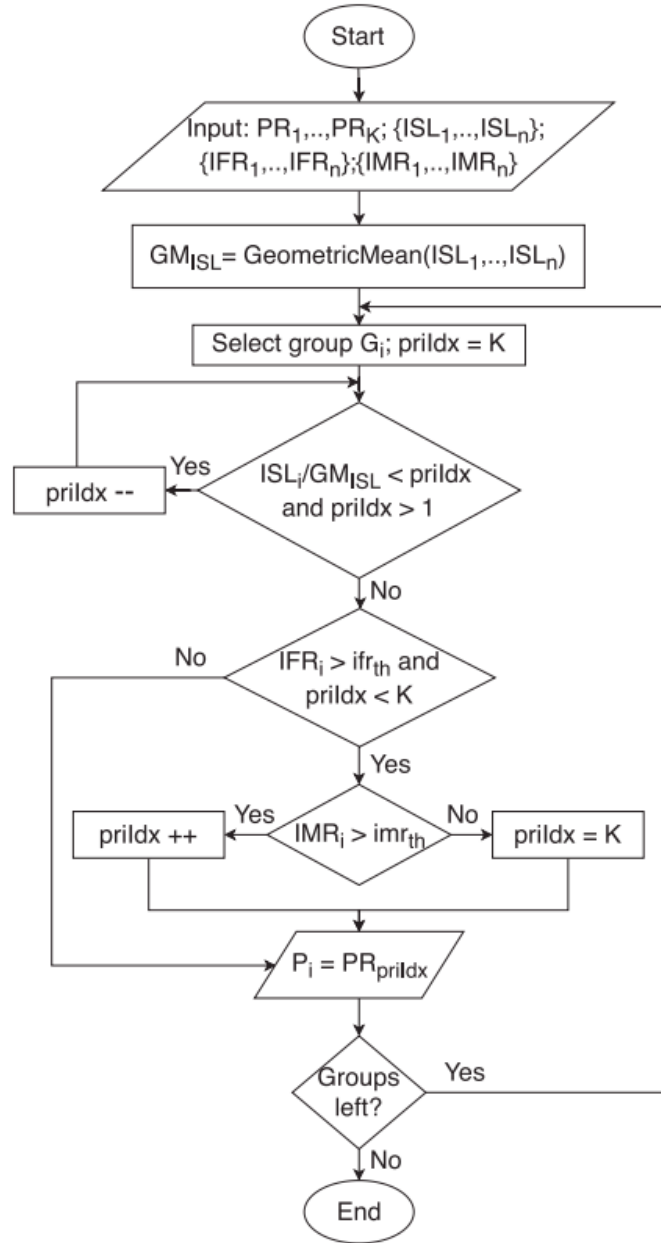


Figure 2: Flowchart of REAL algorithm

5 Implementation

Number of non-LLC instructions and LLC loads is counted at the reorder buffer. Non-LLC stores and LLC stores are counted at store queue at the cpu. The ROB retires the instructions in the program order, any instruction taking more than L2 access time is deemed LLC access. Similarly, store queue removes the store requests in the fixed order. Therefore, any request that takes more than L2 access time is deemed to be an LLC access. We keep a counter at ROB and Store queue, when an instruction is retired if the counter value is greater than threshold then it is counted as LLC access else non-LLC instruction. The number of instruction not accessing LLC is counted for each core. LLC loads is counted for each LLC, core pair. Similarly, stores that do not access LLC are counted for each core and store that access LLC are counted for (LLC, core) pair. We use cacheline address of memory requests to identify the LLC bank at the ROB, store queue.

$$LLC_id = (physical_address/64)\%num_LLCs \quad (4)$$

Instruction fetches at the LLC is counted at the message buffer carrying the requests to the LLCs. Similarly, Instruction fetch misses at LLC is counted at the "requestToDir" message buffer at the directory controller. I_fetch_LLCs and I_miss_LLCs are counted for each (LLC,core) pair.

When the request message reaches the LLC it does contain the information whether it is an instruction fetch or load instruction. When the request reaches the Directory controller it does not contain the information of which core it came from.

To facilitate the I_fetch_LLCs and I_miss_LLCs counting I have added 2 fields in the request message and coherence message structure which specifies core_id and request type to identify core_id and request type at the LLC and directory controller for counting the I_fetch_LLC and I_miss_LCC. These fields are set at the sequencer and are copied between the messages at each controller in the request path.

In this implementation each core is a separate group i.e., group size is 1 because every core is placed at different location and experiences different latencies from the LLCs and DRAM. State machine is run for each LLC slice.

At the start of the system each core is given 2 slots. Counters are accumulated during an interval, at the end of the interval the ISL, IFR, IMR are calculated for each (core, LLC) pair. Priority values for cores are calculated at each LLC using the state machine. Priority values provide the number of slots from the priority vector. The slots are given in consecutive order to the cores assuming core has spacial locality in its memory accesses which leads to row buffer hits at the DRAM. The core with the highest priority is ordered first.

During the interval the given slots are used to service the requests at LLC. At "L2requestToL3cache" message buffer the next core to service is fetched. The slots are searched in the decreasing order of priority until core with slots remaining and having a ready message at LLC is found. If there is no core satisfying the condition then slots are exhausted for the cores whose messages

are available, slots are reset to the initial value.

6 Experiments

I ran micro-benchmarks and SPEC2006 benchmarks using FR-FIFO (default, first ready first in first out) and REAL policy. The microbenchmarks simulation is run completely. Whereas the SPEC2006 benchmarks are run until at least one core executes 100 M instructions after fast forwarding the first 1 B instructions and making cache warm for 20 M instructions. I use the parameters $IFR_{th} = 0.02$ and $IMR_{th} = 0.5$ and $invtervalsiz = 700K$ clock cycles as suggested in [1]. I use the priority vector $\{1,2,3,4,5,6,7,8\}$ to assign the slots to the core. This priority vector is chosen to increase the variation among the priorities given to the cores. The previous REAL implementation used the priority values $\{2,4,6,8\}$.

6.1 Stride micro-benchmark

Stride micro-benchmark involves reading or writing to an array at fixed interval. Stride-1 accesses a new cache line every 8th access, Stride-8 accesses a new cache line every access (cache line size in 64 bytes, can hold 8 4 bytes integer).

Load stride micro-benchmark

```
int main(){
    int j
    int arr[stride*100000];
    j = j+arr[0]+1;
    j = j+arr[stride]+1;
    j = j+arr[stride*2]+1;
    .
    .
    j = j+arr[stride*99999]+1;
}
```

Store stride micro-benchmark

```
int main(){
    int arr[stride*100000];
    int j=0;
    arr[0] = j+1;
    arr[stride] = j+1;
    .
    .
    arr[stride*99999] = j+1;
}
```

6.1.1 Results and analysis

The following table represents the average instructions per cycle of FR-FIFO vs REAL for load stride micro-benchmark when core 0 - core 3 are run the first stride value and core 4 - core 7 run the second stride value.

stride	FRFIFO	REAL
1-8	0.023376	0.027269
2-8	0.022310	0.026125
3-8	0.020914	0.024254
4-8	0.019642	0.021917
5-8	0.018478	0.019543
6-8	0.017480	0.019235
7-8	0.016571	0.018464
8-8	0.015726	0.017278

Table 1: load stride microbenchmark

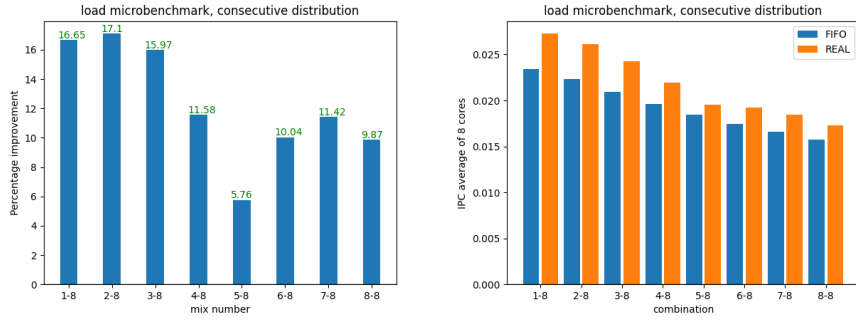


Figure 3: Load micro-benchmark consecutive distribution Left: IPC improvement Right: IPC values

We observe a significant increase in IPC when the ratio of strides is high because the difference in accesses is significant and there is opportunity to improve the performance.

And the performance is better for all same instructions because the slots are allocated consecutively which leads row buffer hits at DRAM.

stride	FRFIFO	REAL
1-8	0.019126	0.019424
2-8	0.013548	0.013634
3-8	0.010023	0.010052
4-8	0.008439	0.008272
5-8	0.006974	0.007084
6-8	0.006256	0.006342
7-8	0.005619	0.005766
8-8	0.005716	0.005442

Table 2: Store microbenchmark IPC.

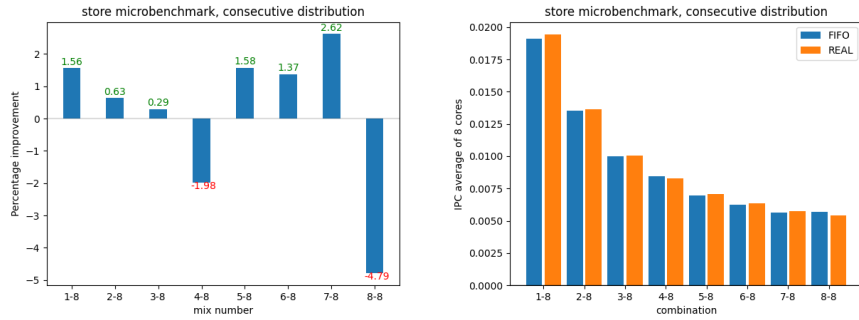


Figure 4: Store micro-benchmark consecutive distribution Left: IPC improvement Right: IPC values

Store benchmarks do not improve significantly because multiple loads are issued in the same clock cycle by the core but stores are issued only after the completion of previous store in the current CPU model. Thus there is not high traffic at LLC and less opportunity for improvement.

Following are when the load strides are distributed alternatively among cores (stride-x is run on cores 0,2,4,6 stride-y is run on cores 1,3,5,7).

stride	FRFIFO	REAL
1-8	0.023314	0.027181
2-8	0.022453	0.026281
3-8	0.020932	0.023925
4-8	0.019611	0.021074
5-8	0.018228	0.019870
6-8	0.017455	0.018827
7-8	0.016568	0.018220

Table 3: Load microbenchmark IPC for alternate distribution.

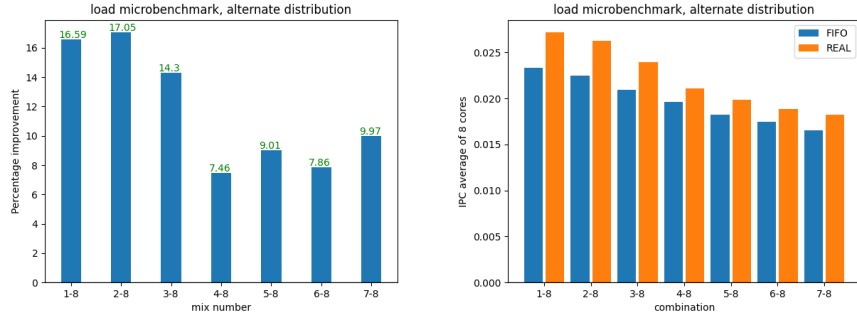


Figure 5: Load micro-benchmark alternate distribution Left: IPC improvement Right: IPC values

stride	FRFIFO IPC	REAL IPC
1-8	0.018508	0.018372
2-8	0.013028	0.013102
3-8	0.010008	0.010023
4-8	0.008407	0.008173
5-8	0.006909	0.007089
6-8	0.006343	0.006270
7-8	0.005856	0.005705

Table 4: Store micro-benchmark IPC for alternate distribution.

We observe very similar behavior when they are assigned consecutively because all the LLCs are accessed equally by stride micro-benchmarks. Thus, changing the position does not have a very large effect. It would have more effect if a core was accessing a particular LLC more than other LLCs.

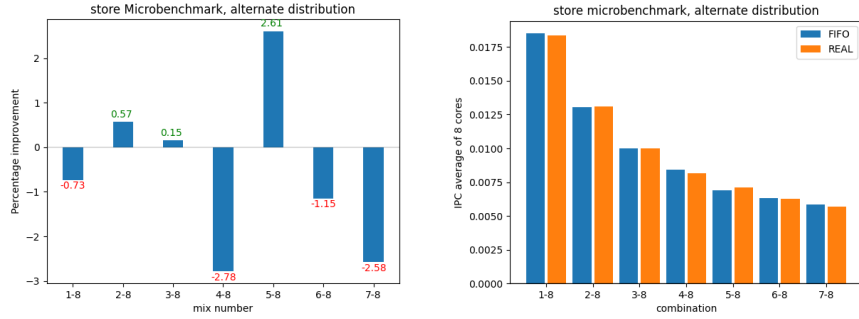


Figure 6: Store micro-benchmark alternate distribution Left: IPC improvement Right: IPC values

6.2 SPEC2006 benchmark mixes

6.2.1 SPEC benchmarks SET 1

		memory intensive		compute intensive		mixed		
Set1	core0	core1	core2	core3	core4	core5	core6	core7
Mix1	garness	garness	garness	soplex	soplex	soplex	omnetpp	omnetpp
Mix2	garness	garness	soplex	soplex	omnetpp	omnetpp	milc	milc
Mix3	soplex	soplex	omnetpp	omnetpp	milc	milc	mcf	lbm
Mix4	soplex	garness	milc	milc	lbm	lbm	mcf	libquantum
Mix5	soplex	soplex	milc	milc	leslie3d	leslie3d	mcf	mcf
Mix6	lbm	lbm	mcf	mcf	leslie3d	leslie3d	milc	milc
Mix7	libquantum	libquantum	milc	milc	leslie3d	leslie3d	mcf	mcf
Mix8	dealII	dealII	astar	astar	sphinx3	sphinx3	GemsFDTD	GemsFDTD
Mix9	bzip2	bzip2	wrf	wrf	gobmk	hmmmer	milc	bwaves
Mix10	gobmk	wrf	hmmmer	gromacs	milc	bwaves	zeusmp	libquantum
Mix11	dealII	h264ref	milc	zeusmp	bwaves	libquantum	leslie3d	lbm
Mix12	dealII	libquantum	libquantum	milc	milc	leslie3d	mcf	mcf
Mix13	hmmmer	hmmmer	gobmk	gobmk	GemsFDTD	GemsFDTD	omnetpp	garness
Mix14	dealII	gromacs	bzip2	hmmmer	omnetpp	garness	soplex	soplex
Mix15	GemsFDTD	GemsFDTD	garness	garness	soplex	soplex	omnetpp	omnetpp
Mix16	dealII	dealII	sjeng	sjeng	omnetpp	omnetpp	milc	milc
Mix17	hmmmer	bzip2	omnetpp	soplex	garness	milc	zeusmp	bwaves
Mix18	dealII	garness	omnetpp	soplex	mcf	lbm	milc	leslie3d

Figure 7: SPEC2006 set 1 benchmark classification

mix	FRFIFO	REAL	mix	FRFIFO	REAL
mix1	0.728767	0.727133	mix10	0.511783	0.525733
mix2	0.633378	0.639339	mix11	0.347399	0.359171
mix3	0.370711	0.385921	mix12	0.323900	0.341802
mix4	0.288242	0.308943	mix13	0.869381	0.865240
mix5	0.180171	0.184850	mix14	0.815852	0.818825
mix6	0.138663	0.142219	mix15	0.809304	0.807065
mix7	0.185514	0.194944	mix16	0.854953	0.856911
mix8	0.992124	0.998553	mix17	0.590774	0.612625
mix9	0.605157	0.621995	mix18	0.425925	0.453499

Table 5: SPEC2006 benchmarks set1 IPC.

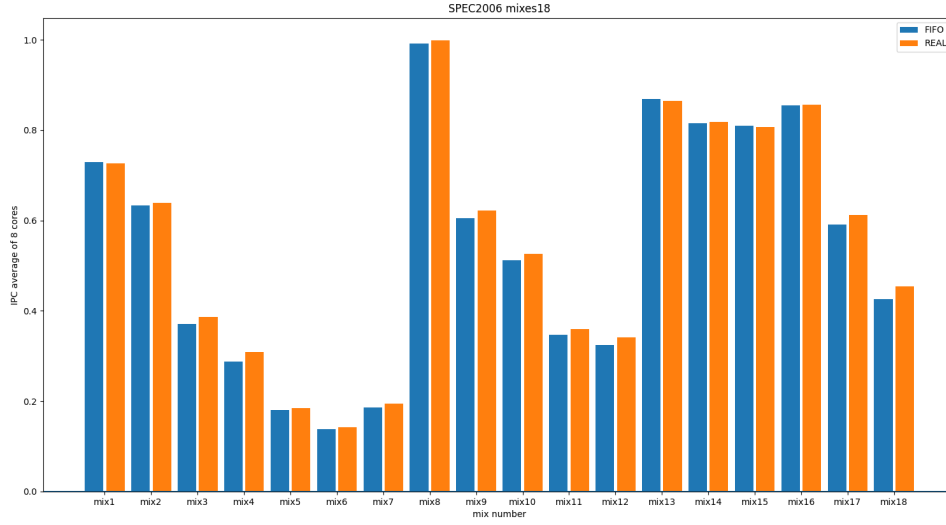


Figure 8: Set1 IPC values

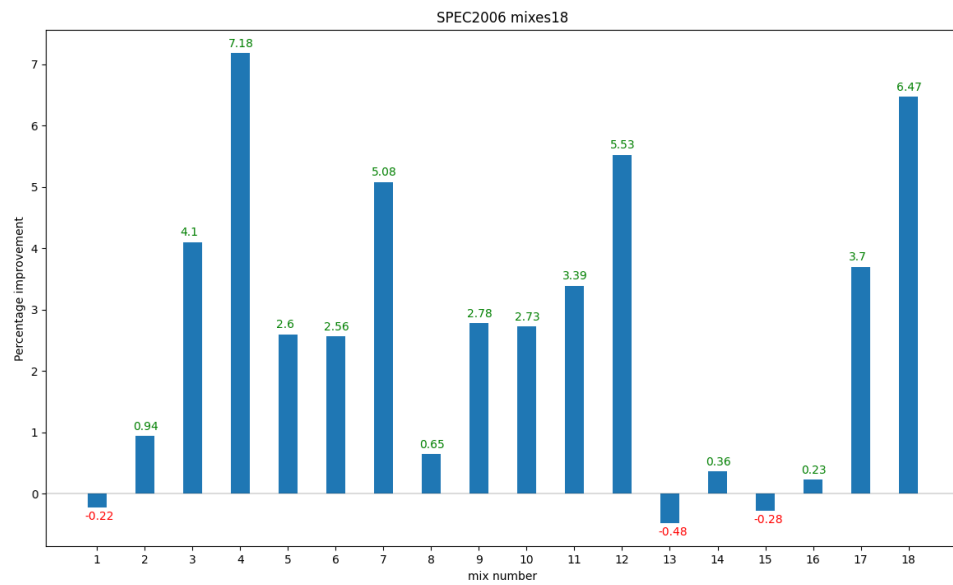


Figure 9: Set1 IPC improvement

- Mix 1,8,13,15 have no memory intensive applications but have some mixed type application which means the number of memory requests are not much to exploit the row buffer hits efficiently. But there are some memory requests which suffer by waiting because others are consuming the slots.
- Mix 2,3,4,5,6,7,9,10,11,12,14,16,17,18 have memory intensive applications, which means more opportunity to improve the performance by facilitating the row buffer hits at the DRAM. The compute intensive benchmark in the mix is given high priority, so it's request do not wait for long time.

6.2.2 SPEC benchmarks SET 2

SET2		memory intensive		compute intensive		mixed		
	core0	core 1	core 2	core3	core4	core5	core6	core7
mix1	GemsFDTD	povray	gromacs	astar	dealII	wrf	astar	GemsFDTD
mix2	GemsFDTD	GemsFDTD	hmmmer	sjeng	namd	astar	dealII	sphinx3
mix3	hmmmer	namd	sjeng	namd	gromacs	gobmk	gromacs	dealII
mix4	bzip2	namd	GemsFDTD	dealII	sphinx3	hmmmer	povray	sphinx3
mix5	soplex	gamess	soplex	soplex	gamess	gamess	soplex	gamess
mix6	omnetpp	gamess	omnetpp	soplex	soplex	gamess	omnetpp	gamess
mix7	soplex	soplex	soplex	omnetpp	omnetpp	gamess	omnetpp	omnetpp
mix8	gamess	soplex	soplex	gamess	gamess	soplex	gamess	omnetpp
mix9	mcf	zeusmp	milc	lbm	lbm	milc	milc	libquantum
mix10	libquantum	mcf	lbm	mcf	bwaves	bwaves	milc	libquantum
mix11	libquantum	milc	lbm	milc	mcf	leslie3d	milc	leslie3d
mix12	leslie3d	mcf	mcf	milc	leslie3d	zeusmp	lbm	lbm
mix13	hmmmer	povray	wrf	gromacs	omnetpp	gamess	soplex	soplex
mix14	dealII	sphinx3	wrf	namd	soplex	soplex	gamess	gamess
mix15	povray	gobmk	hmmmer	astar	gamess	gamess	omnetpp	gamess
mix16	wrf	astar	povray	gromacs	soplex	omnetpp	gamess	soplex
mix17	dealII	gobmk	GemsFDTD	sphinx3	zeusmp	zeusmp	zeusmp	bwaves
mix18	gobmk	povray	gromacs	namd	leslie3d	bwaves	zeusmp	milc
mix19	sphinx3	namd	wrf	povray	lbm	mcf	bwaves	bwaves
mix20	sjeng	wrf	GemsFDTD	namd	libquantum	zeusmp	milc	libquantum
mix21	gamess	soplex	omnetpp	gamess	leslie3d	leslie3d	milc	zeusmp
mix22	omnetpp	gamess	omnetpp	gamess	libquantum	mcf	leslie3d	bwaves
mix23	gamess	soplex	omnetpp	gamess	lbm	milc	bwaves	mcf
mix24	soplex	omnetpp	omnetpp	soplex	milc	mcf	mcf	mcf
mix25	gobmk	gromacs	gamess	gamess	gamess	leslie3d	mcf	mcf
mix26	sphinx3	GemsFDTD	omnetpp	gamess	omnetpp	mcf	milc	zeusmp
mix27	sphinx3	povray	omnetpp	gamess	omnetpp	zeusmp	libquantum	zeusmp
mix28	wrf	bzip2	soplex	omnetpp	gamess	libquantum	lbm	bwaves

Figure 10: SPEC2006 set 2 benchmark classification

mix	FRFIFO	REAL	mix	FRFIFO	REAL
mix1	1.087426	1.084417	mix15	1.080962	1.071858
mix2	1.008358	1.00768	mix16	0.950626	0.93624
mix3	1.01051	1.009903	mix17	0.562228	0.568391
mix4	1.036502	1.036748	mix18	0.55319	0.57643
mix5	0.643643	0.650164	mix19	0.508196	0.519464
mix6	0.82791	0.820598	mix20	0.565256	0.569356
mix7	0.682379	0.680501	mix21	0.443122	0.464921
mix8	0.748701	0.745421	mix22	0.548069	0.587881
mix9	0.228657	0.232358	mix23	0.447755	0.466661
mix10	0.182073	0.190549	mix24	0.331856	0.363255
mix11	0.189777	0.191756	mix25	0.652701	0.656044
mix12	0.131667	0.130776	mix26	0.660956	0.675076
mix13	0.732386	0.734199	mix27	0.813222	0.811459
mix14	0.777048	0.769693	mix28	0.388841	0.403801

Table 6: SPEC2006 benchmark set 2 IPC.

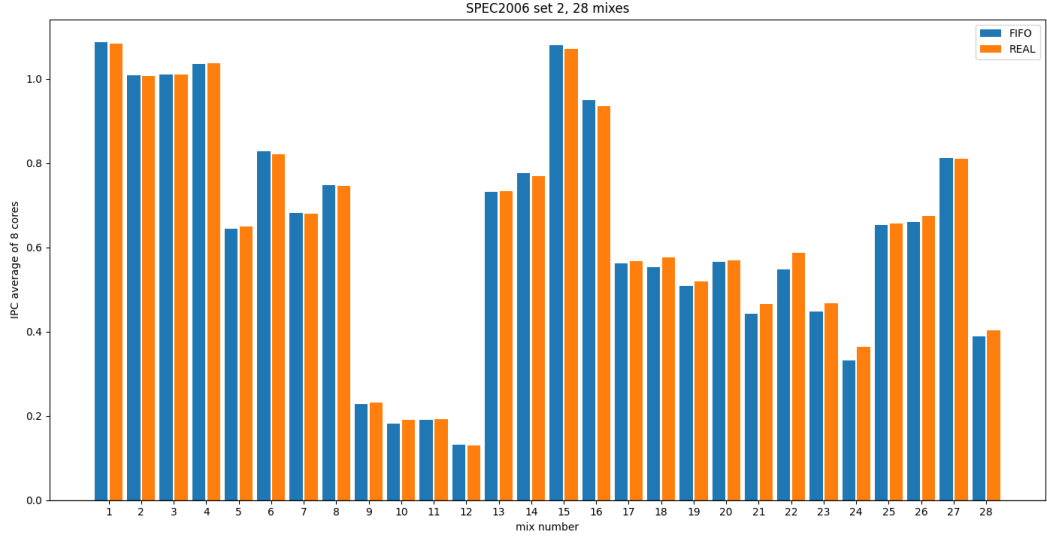


Figure 11: Set 2 IPC values

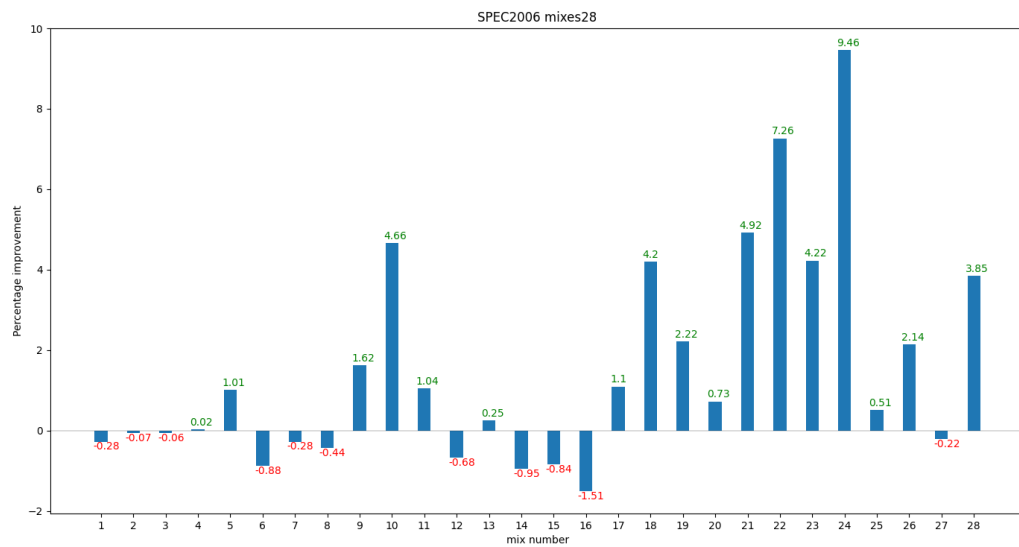


Figure 12: Set 2 IPC improvement

- The REAL performs poorer than the FRFIFO when all the benchmarks are of similar type. When cores get same priority number the cores with lesser core id are prioritized. This increases the waiting time of last cores.
- Here also we observe that the compute intensive mixes do not better because when a multiple request arrive the core with higher priority is favoured and the lesser priority core must have to wait. This effects the compute intensive benchmarks because they send lesser requests to the LLC when those requests are delayed by small amount the IPC is affected.
- We observe good improvement in benchmark which have both memory intensive and compute intensive benchmarks. Because the REAL prioritizes the compute intensive benchmarks. This leads to serving requests of the compute intensive benchmarks on priority, those requests do not have to wait in long queue.

7 Future scope

- The current implementation does not exploit the network topology for assigning the priority values to the Cores. The asymmetry in the network must be studied and should be considered for assigning priorities.
- If two cores have same priority then the REAL prioritizes the core with smaller core id. This may effect the last cores to tackle this the cores with same priority may be rotated every time the slots are reset. Asymmetry in network can be used to break the tie.
- There are multiple independent LLC banks and a single directory controller in the system. This will lead to request queuing at the directory controller. Therefore, some policy must be implemented at the directory so as to improve the row buffer hits at the DRAM and servicing the critical requests on priority.

8 Conclusion

The REAL policy is applicable to SOC with multi bank LLC also. It provides performance improvement when there is mix of compute intensive and memory intensive applications and decrements the performance slightly when all the applications are of similar type. More experiments and analysis has to be done to understand the behavior properly.

References

- [1] Sakshi Tiwari, Shreshth Tuli, Isaar Ahmad, Ayushi Agarwal, Preeti Ranjan Panda, and Sreenivas Subramoney. 2019. REAL: REquest Arbitration in Last Level Caches. *ACM Trans. Embed. Comput. Syst.* 18, 6, Article 115 (November 2019), 24 pages.
- [2] Jason Lowe-Power, Abdul Mutaal Ahmad, Ayaz Akram, Mohammad Alian, Rico Amslinger, Matteo Andreozzi, et al. 2021. The gem5 Simulator: Version 20.0+. *ArXiv Preprint ArXiv:2007.03152*, 2021.